

ATR Trailing Stop Manager - Benutzerhandbuch

Inhaltsverzeichnis

1. [Überblick](#)
 2. [Installation](#)
 3. [Schnellstart](#)
 4. [Kernkonzept](#)
 5. [Konfiguration](#)
 6. [Betriebsmodi](#)
 7. [Praxis-Beispiele](#)
 8. [Optimierung](#)
 9. [Troubleshooting](#)
 10. [Performance-Metriken](#)
-

1. Überblick

Was ist der ATR Trailing Stop Manager?

Ein intelligentes Python-Tool für MetaTrader 5, das Ihre Stop-Loss-Orders automatisch und dynamisch an die Marktvolatilität anpasst, um **maximale Gewinne** bei kontrolliertem Risiko zu erzielen.

Hauptvorteile

-  **Maximale Gewinnmitnahme:** Lässt Gewinner laufen mit intelligentem Trailing
-  **Marktadaptiv:** Passt sich automatisch an die aktuelle Volatilität an
-  **Mehrstufigensystem:** Verschiedene Trailing-Abstände je nach Profit-Level
-  **Parallelbetrieb:** Läuft problemlos neben Ihrem Trading-Bot
-  **Timeframe-optimiert:** Perfekt für 3-15 Minuten Trading

Systemanforderungen

- Windows 10/11 (64-bit)
 - Python 3.8 oder höher
 - MetaTrader 5 Terminal
 - Aktive Internetverbindung
 - Mindestens 4GB RAM
-

2. Installation

Schritt 1: Python-Pakete installieren

```
bash

pip install MetaTrader5 pandas numpy
```

Schritt 2: MetaTrader 5 vorbereiten

1. Algo-Trading aktivieren:

- Tools → Options → Expert Advisors
- ✓ "Allow algorithmic trading"
- ✓ "Allow DLL imports"

2. Python-Integration prüfen:

```
python

import MetaTrader5 as mt5
mt5.initialize()
print(mt5.version())
mt5.shutdown()
```

Schritt 3: Skript einrichten

1. Speichern Sie `atr_trailing_stop.py` in einem Projektordner
 2. Erstellen Sie eine Konfigurationsdatei (optional)
 3. Testen Sie mit: `python atr_trailing_stop.py`
-

3. Schnellstart

Minimale Konfiguration (3 Schritte)

```
python

# 1. Symbol festlegen
SYMBOL = "XAUUSD" # Oder Ihr bevorzugtes Symbol

# 2. Timeframe wählen (M3, M5, oder M15)
ATR_TIMEFRAME = mt5.TIMEFRAME_M5

# 3. Starten
python atr_trailing_stop.py
```

Standard-Einstellungen

Parameter	Standardwert	Beschreibung
ATR Period	14	Lookback-Periode für ATR
Check Interval	30 Sek	Prüfintervall
Initial Stop	1.5x ATR	Anfänglicher Stop-Loss
Min Change	10 Points	Minimum für SL-Update

4. Kernkonzept

Die ATR-Methodik

ATR (Average True Range) misst die durchschnittliche Volatilität über N Perioden.

True Range = MAX(
 High - Low,
 |High - Previous Close|,
 |Low - Previous Close|
)
ATR = Moving Average of True Range

Profit-Stufen-System

Das System arbeitet mit **6 Profit-Stufen**, die den Stop-Loss progressiv nachziehen:

Profit (in ATR)	Aktion	Beschreibung
< 0.5 ATR	Kein Trailing	Position atmen lassen
0.5 - 1.0 ATR	Breakeven	Risiko eliminieren
1.0 - 2.0 ATR	Trail 0.5 ATR	Erste Gewinnsicherung
2.0 - 3.0 ATR	Trail 1.0 ATR	Moderate Sicherung
3.0 - 5.0 ATR	Trail 1.5 ATR	Trend folgen
> 5.0 ATR	Trail 2.0 ATR	Maximum rausholen

Beispiel-Szenario

XAUUSD Buy @ 2050.00

ATR = 2.00 (20 Pips)

Initial SL = 2050.00 - (1.5 * 2.00) = 2047.00

Preis steigt auf 2051.00 (+0.5 ATR):

→ SL auf 2050.00 (Breakeven)

Preis steigt auf 2052.00 (+1.0 ATR):

→ SL auf 2051.00 (Trail 0.5 ATR)

Preis steigt auf 2054.00 (+2.0 ATR):

→ SL auf 2052.00 (Trail 1.0 ATR)

5. Konfiguration

Basis-Parameter

```
python
```

```
# Symbol-Einstellungen
```

```
SYMBOL = "XAUUSD" # Trading-Symbol
```

```
CHECK_INTERVAL_SECONDS = 30 # Prüfintervall
```

```
# ATR-Einstellungen
```

```
ATR_PERIOD = 14 # ATR-Periode (14 = Standard)
```

```
ATR_TIMEFRAME = mt5.TIMEFRAME_M5 # M1, M3, M5, M15, M30
```

```
ATR_SMOOTHING = 1.0 # 1.0 = keine Glättung
```

```
# Risk Management
```

```
INITIAL_STOP_ATR_MULTIPLIER = 1.5 # Initialer Stop bei 1.5x ATR
```

```
MIN_POINTS_TO_UPDATE = 10 # Minimum Points für Update
```

```
MAX_SLIPPAGE = 20 # Max erlaubte Slippage
```

Profit-Level anpassen

```
python
```

```
# Konservativ (für volatile Märkte)
```

```
PROFIT_LEVELS = [  
    {'min_profit_atr': 0.0, 'trail_atr': None},  
    {'min_profit_atr': 1.0, 'trail_atr': 0.0}, # Später Breakeven  
    {'min_profit_atr': 2.0, 'trail_atr': 1.0}, # Weiteres Trailing  
    {'min_profit_atr': 4.0, 'trail_atr': 2.0},  
]
```

```
# Aggressiv (für Trend-Märkte)
```

```
PROFIT_LEVELS = [  
    {'min_profit_atr': 0.0, 'trail_atr': None},  
    {'min_profit_atr': 0.3, 'trail_atr': 0.0}, # Früher Breakeven  
    {'min_profit_atr': 0.5, 'trail_atr': 0.3}, # Enges Trailing  
    {'min_profit_atr': 1.0, 'trail_atr': 0.5},  
    {'min_profit_atr': 2.0, 'trail_atr': 0.8},  
]
```

Volatilitäts-Anpassungen

```
python
```

```
# Volatilitäts-Features
```

```
VOLATILITY_ADJUSTMENT = True # Ein/Aus  
HIGH_VOLATILITY_MULTIPLIER = 1.2 # +20% Abstand bei hoher Vola  
LOW_VOLATILITY_MULTIPLIER = 0.8 # -20% Abstand bei niedriger Vola
```

```
# Zeit-basierte Anpassungen
```

```
TIME_BASED_ADJUSTMENT = True  
WEEKEND_PROTECTION = True # Freitags engere Stops  
NEWS_PROTECTION_MINUTES = 30 # Vor News engere Stops
```

6. Betriebsmodi

Standalone-Betrieb

```
bash
```

```
# Direkt starten
```

```
python atr_trailing_stop.py
```

```
# Mit Custom-Config
```

```
python atr_trailing_stop.py --config my_config.json
```

Parallel zu Trading-Bot

Option 1: Subprocess (Empfohlen)

```
python

import subprocess
import time

# Trading Bot starten
bot_process = subprocess.Popen(['python', 'trading_bot.py'])

# Trailing Stop Manager starten
trail_process = subprocess.Popen(['python', 'atr_trailing_stop.py'])

# Beide laufen parallel
try:
    while True:
        time.sleep(1)
except KeyboardInterrupt:
    bot_process.terminate()
    trail_process.terminate()
```

Option 2: Threading

```
python

import threading
from atr_trailing_stop import main as trailing_main
from trading_bot import main as bot_main

# Als Threads starten
trail_thread = threading.Thread(target=trailing_main, daemon=True)
bot_thread = threading.Thread(target=bot_main, daemon=True)

trail_thread.start()
bot_thread.start()

# Hauptprogramm am Leben halten
trail_thread.join()
```

Option 3: Multiprocessing

```
python
```

```
from multiprocessing import Process
```

```
def run_trailing_stop():
```

```
    import atr_trailing_stop
```

```
    atr_trailing_stop.main()
```

```
def run_trading_bot():
```

```
    import trading_bot
```

```
    trading_bot.main()
```

```
if __name__ == '__main__':
```

```
    p1 = Process(target=run_trailing_stop)
```

```
    p2 = Process(target=run_trading_bot)
```

```
    p1.start()
```

```
    p2.start()
```

```
    p1.join()
```

```
    p2.join()
```

7. Praxis-Beispiele

Beispiel 1: XAUUSD Scalping (M3)

```
python
```

```
# Optimale Settings für Gold-Scalping
```

```
SYMBOL = "XAUUSD"
```

```
ATR_TIMEFRAME = mt5.TIMEFRAME_M3
```

```
ATR_PERIOD = 10 # Schnellere Reaktion
```

```
CHECK_INTERVAL_SECONDS =
```